

IEF2I



E D U C A T I O N

Cahier des charges

Docker et Kubernetes - Virtualisation des applications

BENYAMINA Mohamed
mohamedbenyamina@formateur.ief2i.fr
01/06/2026

Sommaire

1. Contexte du projet	2
2. Présentation du projet	2
3. Objectifs pédagogiques	3
4. Architecture globale attendue	4
5. Technologies autorisées	4
6. Fonctionnalités minimales attendues	5
7. Phase 1 - Infrastructure Docker	5
Objectif	5
8. Livrables attendus — Phase Docker	6
9. Phase 2 - Migration Kubernetes	7
Objectif	7
10. Contraintes Kubernetes	7
11. Haute disponibilité et résilience	8
12. Observabilité et supervision	8
13. Sécurité	8
14. Livrables attendus - Phase Kubernetes	9
15. Extensions avancées	9
16. Organisation du travail	9
17. Soutenance finale	10
18. Critères d'évaluation	10
19. Conclusion	11

1. Contexte du projet

Dans le cadre du module « Docker & Kubernetes - Virtualisation des applications », les étudiants devront concevoir, développer, conteneuriser puis orchestrer une plateforme SaaS moderne reposant sur une architecture microservices.

Ce projet pédagogique a pour objectif de reproduire les contraintes et méthodologies rencontrées dans un environnement professionnel DevOps et Cloud Native. Il ne s'agit pas uniquement de produire une application fonctionnelle, mais également de mettre en œuvre une véritable infrastructure applicative industrialisée, supervisée et scalable.

Le projet sera réalisé par groupe de deux à trois étudiants maximum et se déroulera en deux grandes phases complémentaires. La première phase portera sur la conception de l'infrastructure Docker tandis que la seconde consistera à migrer cette architecture vers un environnement Kubernetes orchestré.

L'intégralité du travail réalisé durant la phase Docker devra être réutilisée dans la phase Kubernetes. L'objectif est de faire évoluer progressivement une infrastructure conteneurisée classique vers une plateforme cloud native complète.

2. Présentation du projet

Le projet porte sur la conception d'une plateforme nommée « GreenOps Platform ».

Cette plateforme devra permettre la supervision et l'analyse de métriques énergétiques à travers une interface web moderne. Les utilisateurs devront pouvoir consulter des tableaux de bord, surveiller des indicateurs en temps réel, recevoir des alertes et exploiter différentes données collectées par les services de la plateforme.

Au-delà des fonctionnalités métier, l'objectif principal du projet repose sur l'industrialisation de l'infrastructure et l'application des bonnes pratiques DevOps modernes. Une attention particulière sera accordée à l'architecture logicielle, à la conteneurisation, à l'observabilité, à la résilience des services ainsi qu'à la qualité globale du déploiement.

3. Objectifs pédagogiques

À travers ce projet, les étudiants devront démontrer leur capacité à concevoir une architecture distribuée cohérente et à mettre en œuvre une infrastructure conteneurisée complète.

Le projet devra permettre de valider la maîtrise des concepts suivants :

- architecture microservices ;
- création et optimisation de Dockerfiles ;
- orchestration locale avec Docker Compose ;
- gestion des réseaux et volumes Docker ;
- industrialisation des déploiements ;
- monitoring et observabilité ;
- intégration continue et déploiement continu ;
- orchestration Kubernetes ;
- haute disponibilité et scalabilité ;
- sécurisation des infrastructures conteneurisées.

Les choix techniques réalisés devront être justifiés et documentés tout au long du projet.

4. Architecture globale attendue

La plateforme devra être construite autour d'une architecture microservices moderne. Chaque composant devra être isolé dans son propre conteneur et communiquer avec les autres services à travers des mécanismes réseau adaptés.

L'architecture minimale attendue devra comporter une interface frontend, une API Gateway, un service d'authentification, plusieurs services métiers, une base de données PostgreSQL, un système de cache Redis, un reverse proxy ainsi qu'une stack complète de monitoring composée notamment de Prometheus et Grafana.

L'ensemble devra être conçu de manière modulaire afin de faciliter les évolutions futures et la migration vers Kubernetes.

Des schémas d'architecture clairs devront être produits afin d'illustrer l'organisation générale de la plateforme et les communications inter-services.

5. Technologies autorisées

Les étudiants pourront choisir librement leur stack backend parmi Node.js, PHP, Python ou Go, à condition de respecter une architecture microservices cohérente.

Le frontend devra être développé avec React ou Vue.js.

L'utilisation de PostgreSQL sera obligatoire pour la persistance des données. Redis devra être utilisé comme système de cache ou de stockage temporaire.

Pour la gestion du reverse proxy, les étudiants pourront utiliser Nginx ou Traefik.

La supervision devra impérativement reposer sur Prometheus et Grafana.

Enfin, un pipeline CI/CD minimal devra être mis en place à l'aide de GitHub Actions.

6. Fonctionnalités minimales attendues

La plateforme devra proposer un système d'authentification sécurisé permettant la gestion des utilisateurs ainsi que des rôles applicatifs. L'authentification devra être réalisée à l'aide de JWT.

L'interface utilisateur devra permettre la visualisation de différentes métriques énergétiques sous forme de graphiques et tableaux de bord dynamiques.

Un système d'alertes devra également être mis en place afin de détecter certains seuils critiques et de conserver un historique des événements générés.

L'ensemble des services devra exposer des métriques exploitables par Prometheus afin de permettre une supervision globale de la plateforme.

Une attention particulière devra être portée à la qualité des logs produits par les applications et les différents services de l'infrastructure.

7. Phase 1 - Infrastructure Docker

Objectif

La première phase du projet consistera à concevoir une infrastructure Docker complète permettant d'exécuter l'ensemble de la plateforme dans un environnement conteneurisé cohérent et industrialisé.

Les étudiants devront produire des Dockerfiles optimisés pour chacun des services applicatifs. Les images produites devront respecter les bonnes pratiques modernes de conteneurisation, notamment en matière de sécurité, de performance et de taille des images.

L'infrastructure devra être centralisée via Docker Compose. Le fichier de composition devra permettre le déploiement automatique de tous les services nécessaires au fonctionnement de la plateforme.

Les réseaux Docker devront être correctement segmentés afin d'isoler les différents composants de l'infrastructure. Les volumes devront garantir la persistance des données critiques, notamment pour PostgreSQL et Grafana.

La configuration des services devra être externalisée via des variables d'environnement et des fichiers adaptés.

Un reverse proxy devra être mis en place afin de centraliser les accès à la plateforme et de gérer le routage des requêtes.

La stack devra également intégrer un système de monitoring opérationnel reposant sur Prometheus et Grafana.

Enfin, un pipeline CI/CD minimal devra permettre l'automatisation des opérations principales du projet telles que les tests ou la génération des images Docker.

8. Livrables attendus - Phase Docker

À l'issue de cette première phase, chaque groupe devra fournir un dépôt GitHub proprement structuré contenant l'ensemble du projet ainsi qu'une documentation technique détaillée.

Le dépôt devra inclure les Dockerfiles, le fichier Docker Compose, les fichiers de configuration nécessaires, une procédure complète d'installation et de lancement ainsi qu'une documentation expliquant les choix techniques réalisés.

Une démonstration fonctionnelle de l'infrastructure sera également demandée.

9. Phase 2 - Migration Kubernetes

Objectif

La seconde phase du projet consistera à transformer l'infrastructure Docker précédemment réalisée en une véritable plateforme orchestrée sous Kubernetes.

L'objectif sera d'introduire les notions de scalabilité, de haute disponibilité, de résilience et d'automatisation des déploiements dans un environnement distribué moderne.

L'architecture existante devra être réutilisée et adaptée au fonctionnement d'un cluster Kubernetes.

10. Contraintes Kubernetes

L'orchestration devra reposer sur l'utilisation des principaux objets Kubernetes modernes.

Les applications devront être déployées via des Deployments et exposées à travers des Services et un Ingress Controller.

La configuration devra être externalisée à l'aide de ConfigMaps et Secrets.

Les données persistantes devront être gérées à l'aide de Persistent Volumes et Persistent Volume Claims.

Les étudiants devront également mettre en œuvre des mécanismes de résilience via des probes Kubernetes ainsi qu'un système de scaling automatique avec Horizontal Pod Autoscaler.

Une organisation logique du cluster devra être mise en place grâce aux Namespaces.

11. Haute disponibilité et résilience

L'infrastructure Kubernetes devra être capable de supporter la réplication des services et de limiter les interruptions en cas de panne.

Les mécanismes natifs de Kubernetes devront être exploités afin de garantir le redémarrage automatique des applications, la répartition de charge ainsi que la continuité de service.

Une démonstration de résilience et de scaling devra être réalisée lors de la soutenance finale.

12. Observabilité et supervision

La plateforme devra intégrer une stack complète de supervision permettant le suivi du cluster Kubernetes ainsi que des applications déployées.

Les étudiants devront être capables de collecter et d'exploiter les métriques des différents services afin d'analyser le comportement global de l'infrastructure.

Des tableaux de bord Grafana pertinents devront être proposés afin de visualiser l'état de santé du système.

13. Sécurité

Une attention particulière devra être accordée à la sécurisation de l'infrastructure.

Les secrets applicatifs ne devront jamais être stockés directement dans les dépôts Git. Les mécanismes natifs de Kubernetes devront être utilisés afin de protéger les données sensibles.

Les permissions devront être limitées autant que possible et les communications inter-services devront être correctement isolées.

14. Livrables attendus - Phase Kubernetes

Les étudiants devront fournir l'ensemble des manifests Kubernetes nécessaires au déploiement complet de la plateforme.

Une documentation détaillée expliquant l'architecture du cluster, les procédures de déploiement et les mécanismes de supervision devra être produite.

Une démonstration complète du fonctionnement de la plateforme orchestrée sera exigée lors de la soutenance finale.

15. Extensions avancées

Les groupes souhaitant aller plus loin pourront intégrer des fonctionnalités supplémentaires telles qu'un système temps réel via WebSockets, un message broker comme RabbitMQ ou Kafka, une approche GitOps avec ArgoCD, une centralisation des logs ou encore des outils avancés de tests de charge.

Ces fonctionnalités ne sont pas obligatoires mais pourront être valorisées dans l'évaluation finale du projet.

16. Organisation du travail

Le projet devra être réalisé de manière collaborative avec une gestion de version rigoureuse à travers GitHub.

Chaque groupe devra maintenir un historique Git cohérent, documenter régulièrement ses avancées et préparer une soutenance technique structurée.

Une répartition claire des responsabilités au sein de l'équipe est fortement recommandée.

17. Soutenance finale

Chaque groupe devra présenter son architecture technique, justifier ses choix d'implémentation et réaliser une démonstration complète de la plateforme.

La soutenance devra notamment inclure :

- la présentation de l'architecture Docker ;
- la migration Kubernetes ;
- le monitoring ;
- le scaling ;
- la résilience de l'infrastructure ;
- les mécanismes de sécurité mis en œuvre.

Une attention particulière sera portée à la qualité des explications techniques et à la maîtrise globale de l'environnement DevOps présenté.

18. Critères d'évaluation

L'évaluation prendra en compte la qualité globale de l'architecture, la cohérence des choix techniques, la maîtrise de Docker et Kubernetes, la qualité de l'industrialisation, la supervision, la documentation, la stabilité de la plateforme ainsi que la qualité de la soutenance finale.

La capacité des étudiants à appliquer des bonnes pratiques professionnelles modernes sera un élément central de l'évaluation.

19. Conclusion

Ce projet a pour objectif de rapprocher les étudiants des problématiques réelles rencontrées dans les environnements DevOps modernes.

À travers la conception d'une infrastructure complète, scalable et supervisée, les étudiants devront démontrer leur capacité à concevoir des architectures distribuées robustes et industrialisées.

L'ensemble du projet devra refléter une compréhension approfondie des principes de conteneurisation, d'orchestration et d'ingénierie Cloud Native.